

REEMAN Dispatch Service Third-Party API

1.General Conventions

1.1 Base Request URL

All URLs in this document are relative paths behind the Nginx proxy. Build the actual request URL as follows:

Plain Text

```
1 http(s)://{ip-or-domain}/api/reeman-dispatch-service{URL-from-this-document}
```

Example:

- Document URL: POST /task-template/generic/chain
- Actual request: POST <http://192.168.1.2/api/reeman-dispatch-service/task-template/generic/chain>

1.2 Headers

Header	Required	Description
Content-Type: application/json	Required for JSON request bodies	Used by task creation and cancellation APIs
Accept-Language	No	Controls message in standard responses and dictionary description ; supported values include zh-CN , en-US , th-TH

1.3 Date and Time

Type	Format	Example
Date	yyyy-MM-dd	2026-04-21
Date time	yyyy-MM-dd HH:mm:ss	2026-04-21 10:15:00
Boolean	true / false	true

1.4 Standard Success Response

JSON

```
1 {
2   "code": 0,
3   "message": "success",
4   "data": {}
5 }
```

`data` is usually omitted when there is no business data.

1.5 Standard Failure Response

Business failures usually return HTTP `400`:

JSON

```
1 {
2   "code": 5,
3   "message": "Invalid request parameters"
4 }
```

Unexpected errors usually return HTTP `500`:

JSON

```
1 {  
2   "code": 4,  
3   "message": "Internal server error. Check service logs."  
4 }
```

Common business codes:

code	Description
0	Success
1	Duplicate data
2	Missing data
3	Parse failure
4	Service exception
5	Invalid data
6	Operation failed
7	External service error
401	Invalid state. The request is not allowed in the current state

1.6 Integration Notes

- Before creating tasks, the third-party system should confirm that the project, map, point or area, action type, and AGV type already exist on the server.
- A successful task creation response only means the task record has been created or submitted to dispatch. Query task status and action records to confirm execution progress and completion.

2.Recommended Call Flow

1. Query base data: `GET /project/infos`, `GET /map/list/details/{projectId}`, `GET /map/point/list/{map}`, `GET /map/area/list/{map}`.
2. Query dictionaries: `GET /action-type/list-all`, `GET /task/type/list-all`, `GET /agv-type/list-all`.
3. Create tasks: Prefer `POST /task-template/generic/chain`; if templates are preconfigured on site, use `POST /task-template/generic/task-fast/{id}` or `POST /task-template/generic/button-task`.
4. Query tasks: Use `GET /task-record/page` for task records and `GET /action-record/list/{id}` for action details.
5. Cancel tasks: Use `POST /task-record/cancel/{id}`; use `POST /task-record/all-cancel/{projectId}` with caution.
6. Query errors and statistics: Call error record, task statistics, and transport statistics APIs as needed.

3.Base Data and Dictionaries

3.1 Query Project List

- URL: `GET /project/infos`
- Purpose: Get project IDs that can be used to create tasks.
- Request example:

HTTP

```
1 GET /project/infos
```

- Success response example:

JSON

```
1 {  
2   "code": 0,  
3   "message": "success",  
4   "data": [  
5     {  
6       "id": 1,  
7       "projectName": "Test Project A"  
8     }  
9   ]  
10 }
```

- Failure notes: Returns an empty array when there are no projects; query exceptions return HTTP `500`, `code=4`.

3.2 Query Project Map List

- URL: `GET /map/list/details/{projectId}`
- Purpose: Get maps under the project that can be used as task targets.
- Request example:

HTTP

```
1 GET /map/list/details/1
```

- Success response example:

JSON

```
1 {  
2   "code": 0,  
3   "message": "success",  
4   "data": [  
5     {  
6       "id": 1,  
7       "projectId": 1,  
8       "map": "dbe8312c60cbc75967d8b281bf4688ef",  
9       "alias": "1F Main Map",  
10      "floor": 1  
11    }  
12  ]  
13 }
```

- Failure notes: If `projectId` is not numeric, returns HTTP `500`, `code=4`; returns an empty array when no maps exist.

3.3 Query Map Points

- URL: `GET /map/point/list/{map}`
- Purpose: Get point names in a map for task action `targetPoint`.
- Request parameters:

Name	Location	Type	Required	Description
<code>type</code>	query	string	No	Point type, such as <code>pallet</code> , <code>pallet_docking</code> , <code>delivery</code> , <code>charge</code>

- Request example:

HTTP

```
1 GET /map/point/list/dbe8312c60cbc75967d8b281bf4688ef?type=delivery
```

- Success response example:

JSON

```
1 {
2   "code": 0,
3   "message": "success",
4   "data": [
5     {
6       "id": 11,
7       "map": "dbe8312c60cbc75967d8b281bf4688ef",
8       "name": "P_DELIVERY_01",
9       "type": "delivery",
10      "position": [1.0, 2.0, 0.0]
11    }
12  ]
13 }
```

- Failure notes: Returns an empty array when no point matches; parameter format errors return HTTP 500 , code=4 .

3.4 Query Map Areas

- URL: GET /map/area/list/{map}
- Purpose: Get area names in a map for area pickup or area unload action targetArea .
- Request parameters:

Name	Location	Type	Required	Description
type	query	string	No	Area type, such as STORAGE_LOCA TION

- Request example:

HTTP

```
1 GET /map/area/list/dbe8312c60cbc75967d8b281bf4688ef?type=STORAGE_LOCATION
```

- Success response example:

JSON

```
1 {
2   "code": 0,
3   "message": "success",
4   "data": [
5     {
6       "id": 21,
7       "map": "dbe8312c60cbc75967d8b281bf4688ef",
8       "name": "STORE_A",
9       "type": "STORAGE_LOCATION"
10    }
11  ]
12 }
```

- Failure notes: Returns an empty array when no area matches; parameter format errors return HTTP `500`, `code=4`.

3.5 Query Action Types

- URL: `GET /action-type/list-all`
- Purpose: Get task action types. Common third-party values include `PICKUP`, `UNLOAD`, `DELIVERY`, `AREA_PICKUP`, `AREA_UNLOAD`, `CHARGE`.
- Request example:

HTTP

```
1 GET /action-type/list-all
```

- Success response example:

JSON

```
1 {
2   "code": 0,
3   "message": "success",
4   "data": [
5     {
6       "type": "PICKUP",
7       "description": "Pickup"
8     },
9     {
10      "type": "UNLOAD",
11      "description": "Unload"
12    }
13  ]
14 }
```

- Failure notes: This API only reads built-in dictionaries and usually does not produce business failures.

3.6 Query Task Types

- URL: `GET /task/type/list-all`
- Purpose: Query task source types for understanding the type field in task records.
- Request example:

HTTP

```
1 GET /task/type/list-all
```

- Success response example:

JSON

```
1 {
2   "code": 0,
3   "message": "success",
4   "data": [
5     {
6       "type": "FAST",
7       "description": "Fast task"
8     },
9     {
10      "type": "TEMP_TASK_CHAIN",
11      "description": "Temporary task"
12    },
13    {
14      "type": "BUTTON_TASK",
15      "description": "Button task"
16    }
17  ]
18 }
```

- Failure notes: This API only reads built-in dictionaries and usually does not produce business failures.

3.7 Query AGV Types

- URL: `GET /agv-type/list-all`
- Purpose: Query robot types that can be used as task constraints for `agvTypes` .
- Request example:

HTTP

```
1 GET /agv-type/list-all
```

- Success response example:

JSON

```
1 {  
2   "code": 0,  
3   "message": "success",  
4   "data": [  
5     {  
6       "type": "FORK-01",  
7       "length": 1450,  
8       "width": 860,  
9       "kinematic": "DIFF"  
10    }  
11  ]  
12 }
```

- Failure notes: Usually does not produce business failures; query exceptions return HTTP `500`, `code=4`.

3.8 Query Available AGVs

- URL: `GET /agv/page`
- Purpose: Query AGVs under a project, used by third-party systems to specify `agvId` or check online status.
- Request parameters:

Name	Location	Type	Required	Description
<code>page</code>	query	long	No	Default <code>1</code>
<code>size</code>	query	long	No	Default <code>10</code>
<code>projectId</code>	query	long	Yes	Project ID
<code>id</code>	query	string	No	Fuzzy query by AGV ID
<code>type</code>	query	string	No	AGV type
<code>online</code>	query	boolean	No	Online status

- Request example:

HTTP

```
1 GET /agv/page?page=1&size=10&projectId=1&online=true
```

- Success response example:

JSON

```
1 {
2   "code": 0,
3   "message": "success",
4   "data": {
5     "records": [
6       {
7         "id": "AGV-001",
8         "projectId": 1,
9         "type": "FORK-01",
10        "online": true,
11        "lowPower": 20,
12        "taskPower": 80,
13        "crossFloor": true,
14        "standbyPointType": "charge"
15      }
16    ],
17    "total": 1,
18    "size": 10,
19    "current": 1
20  }
21 }
```

- Failure notes: Pagination, project ID, or boolean parameter type errors return HTTP `500`, `code=4`; returns an empty page when no AGV matches.

3.9 Query Point Types

- URL: `GET /point/type/list-all`

- Purpose: Query point type dictionary. Third-party systems can use it to understand `type` returned by `/map/point/list/{map}` .
- Request example:

HTTP

```
1 GET /point/type/list-all
```

- Success response example:

JSON

```
1 {
2   "code": 0,
3   "message": "success",
4   "data": [
5     {
6       "type": "pallet",
7       "description": "Pallet point"
8     },
9     {
10      "type": "delivery",
11      "description": "Delivery point"
12    },
13    {
14      "type": "charge",
15      "description": "Charging station"
16    }
17  ]
18 }
```

- Failure notes: This API only reads built-in dictionaries and usually does not produce business failures.

3.10 Query Area Types

- URL: `GET /area-type/list-all`

- Purpose: Query area type dictionary. Third-party systems can use it to understand `type` returned by `/map/area/list/{map}` .
- Request example:

HTTP

```
1 GET /area-type/list-all
```

- Success response example:

JSON

```
1 {
2   "code": 0,
3   "message": "success",
4   "data": [
5     {
6       "type": "A1",
7       "description": "One-way area"
8     },
9     {
10      "type": "STORAGE_LOCATION",
11      "description": "Storage area"
12    }
13  ]
14 }
```

- Failure notes: This API only reads built-in dictionaries and usually does not produce business failures.

4.Task Creation

4.1 Create Temporary Task Chain (Recommended)

- URL: `POST /task-template/generic/chain`
- Content-Type: `application/json`

- Purpose: The third-party system creates a task by directly submitting an action chain. This does not depend on preconfigured templates.

Request parameters:

Name	Type	Required	Description
<code>projectId</code>	long	Yes	Project ID
<code>agvId</code>	string	No	Specific AGV to execute the task; at least one of <code>agvId</code> and <code>agvTypes</code> is required
<code>agvTypes</code>	string[]	No	Allowed AGV types; at least one of <code>agvId</code> and <code>agvTypes</code> is required
<code>taskChain</code>	array	Yes	Action chain. Must not be empty

Fields of each `taskChain` item:

Field	Type	Required	Description
<code>targetMap</code>	string	Yes	Target map name
<code>targetPoint</code>	string	Yes	Target point. Use an empty string for area actions
<code>targetArea</code>	string	Yes	Target area. Use an empty string for point actions
<code>action</code>	string	Yes	Action type, such as <code>PICKUP</code> , <code>UNLOAD</code> , <code>AREA_PICKUP</code> , <code>AREA_UNLOAD</code>
<code>params</code>	object	No	Action parameters. Keys with <code>null</code> values are not allowed

`params` rules:

- For common `PICKUP`, `UNLOAD`, `DELIVERY`, `AREA_PICKUP`, `AREA_UNLOAD`, and `CHARGE` tasks, usually pass `{}` or omit `params`.

- `params` must be a JSON object. Key names are case-sensitive. Values can be string, number, integer, boolean, array, or object, but must not be `null`.
- Storage coordinates, docking coordinates, path rotation, door control, elevator, and similar parameters are generated by the dispatch service from map and point configuration. Third-party systems should not manually pass these internal parameters when creating tasks.
- If business parameters need to be passed through to the robot action protocol, field names, data types, and units must be agreed upon in advance.

No-extra-parameter example:

JSON

```
1 {  
2   "params": {}  
3 }
```

Protocol-agreed extension parameter example:

JSON

```
1 {  
2   "params": {  
3     "externalOrderNo": "WMS-202605270001",  
4     "priority": 1,  
5     "needReceipt": true  
6   }  
7 }
```

Invalid example:

JSON

```
1 {  
2   "params": {  
3     "externalOrderNo": null  
4   }  
5 }
```

Third-party systems usually only need the following types:

Scenario	Parameter Key	JSON Type	Example	Description
No extra parameter	None	object	<code>{}</code>	Recommended default value
Business order number passthrough	<code>externalOrderNo</code>	string	<code>"WMS-202605270001"</code>	Example field. Must be agreed upon by both parties
Priority passthrough	<code>priority</code>	integer	<code>1</code>	Example field. Must be agreed upon by both parties
Boolean flag passthrough	<code>needReceipt</code>	boolean	<code>true</code>	Example field. Must be agreed upon by both parties
Delivery countdown	<code>DELIVERY_COUNTDOWN</code>	integer	<code>30</code>	Only pass when supported by the robot action protocol. Unit follows the on-site protocol

Point task request example:

JSON

```
1 {
2   "projectId": 1,
3   "agvId": "AGV-001",
4   "agvTypes": null,
5   "taskChain": [
6     {
7       "targetMap": "dbe8312c60cbc75967d8b281bf4688ef",
8       "targetPoint": "P_PICK_01",
9       "targetArea": "",
10      "action": "PICKUP",
11      "params": {}
12    },
13    {
14      "targetMap": "dbe8312c60cbc75967d8b281bf4688ef",
15      "targetPoint": "P_PUT_01",
16      "targetArea": "",
17      "action": "UNLOAD",
18      "params": {}
19    }
20  ]
21 }
```

Auto-assign by AGV type request example:

JSON

```
1 {
2   "projectId": 1,
3   "agvId": null,
4   "agvTypes": ["FORK-01"],
5   "taskChain": [
6     {
7       "targetMap": "dbe8312c60cbc75967d8b281bf4688ef",
8       "targetPoint": "",
9       "targetArea": "STORE_A",
10      "action": "AREA_PICKUP",
11      "params": {}
12    },
13    {
14      "targetMap": "dbe8312c60cbc75967d8b281bf4688ef",
15      "targetPoint": "",
16      "targetArea": "STORE_B",
17      "action": "AREA_UNLOAD",
18      "params": {}
19    }
20  ]
21 }
```

Success response example:

JSON

```
1 {
2   "code": 0,
3   "message": "success",
4   "data": {
5     "taskRecordId": 2001
6   }
7 }
```

Failure notes:

- code=5 : agvId and agvTypes are both empty, taskChain is empty, action fields are invalid, or params contains a null value.

- `code=2` : Project, map, point, area, AGV, or AGV type does not exist.
- `code=401` : Project is not running, AGV is unavailable, the first target is occupied, current state does not allow task creation, or route planning failed.
- HTTP `500` , `code=4` : Request body structure is invalid or an internal service error occurred.

4.2 Create Task from Template

- URL: `POST /task-template/generic/task-fast/{id}`
- Content-Type: `application/json`
- Purpose: Create a task from a server-side preconfigured task template. Suitable when the third-party system only passes a template ID and optional AGV ID.

Request parameters:

Name	Location	Type	Required	Description
<code>id</code>	path	long	Yes	Task template ID
<code>appoint</code>	body	string	No	Specific AGV ID to execute the task

Request example:

HTTP

```

1 POST /task-template/generic/task-fast/100
2 Content-Type: application/json

```

JSON

```

1 {
2   "appoint": "AGV-001"
3 }

```

Success response example:

JSON

```
1 {  
2   "code": 0,  
3   "message": "success"  
4 }
```

Failure notes:

- `code=2` : Template does not exist.
- `code=401` : Project, AGV, target point, or task state does not allow task creation.
- `code=5` : Specified AGV or template action data is invalid.

4.3 Create Task by Button Number

- URL: `POST /task-template/generic/button-task`
- Content-Type: `application/json`
- Purpose: Create a task from a template bound to a button number. Suitable for third-party systems integrating with on-site buttons or call points.

Request parameters:

Name	Type	Required	Description
<code>projectId</code>	long	Yes	Project ID
<code>buttonKey</code>	string	Yes	Button number
<code>appoint</code>	string	No	Specific AGV ID to execute the task

Request example:

JSON

```
1 {  
2   "projectId": 1,  
3   "buttonKey": "BTN001",  
4   "appoint": "AGV-001"  
5 }
```

Success response example:

JSON

```
1 {  
2   "code": 0,  
3   "message": "success"  
4 }
```

Failure notes:

- `code=2` : No template binding found for the button.
- `code=5` : `buttonKey` is empty.
- `code=401` : The same template has unfinished actions, the first target is occupied, or current state does not allow task creation.

4.4 Query Available Templates

- URL: `GET /task-template/page`
- Purpose: Query template IDs under a project before creating tasks from templates.

Request parameters:

Name	Location	Type	Required	Description
page	query	long	No	Default 1
size	query	long	No	Default 10
projectId	query	long	Yes	Project ID
name	query	string	No	Fuzzy query by template name
description	query	string	No	Fuzzy query by description

Request example:

HTTP

1 GET /task-template/page?page=1&size=10&projectId=1&name=delivery

Success response example:

JSON

```
1 {
2   "code": 0,
3   "message": "success",
4   "data": {
5     "records": [
6       {
7         "id": 100,
8         "name": "Pickup then inspection delivery",
9         "description": "For first-floor sample delivery",
10        "buttonKey": "BTN001"
11      }
12    ],
13    "total": 1,
14    "size": 10,
15    "current": 1
16  }
17 }
```

Failure notes: Pagination or project ID parameter type errors return HTTP `500`, `code=4`; returns an empty page when no template matches.

4.5 Query Template Details

- URL: `GET /task-template/info/{id}`
- Purpose: Confirm the template action chain before creating a task from a template.

Request example:

HTTP

```
1 GET /task-template/info/100
```

Success response example:

JSON

```
1 {  
2   "code": 0,  
3   "message": "success",  
4   "data": {  
5     "id": 100,  
6     "name": "Pickup then inspection delivery",  
7     "supportRobotTypes": ["FORK-01"],  
8     "taskActionList": [  
9       {  
10        "id": 1,  
11        "targetMap": "dbe8312c60cbc75967d8b281bf4688ef",  
12        "targetPoint": "P_PICK_01",  
13        "targetArea": "",  
14        "action": "PICKUP"  
15      }  
16    ]  
17  }  
18 }
```

Failure notes:

- `code=2`: Template does not exist.
- `id` type errors return HTTP `500`, `code=4`.

5.Task Query

5.1 Query Task Records

- URL: `GET /task-record/page`
- Purpose: Query task execution status. After creating a task, the third-party system should query by returned `taskRecordId` or other business filters.

Request parameters:

Name	Location	Type	Required	Description
page	query	long	No	Default 1
size	query	long	No	Default 10
projectId	query	long	Yes	Project ID
status	query	string	No	Task status
name	query	string	No	Task name
description	query	string	No	Task description

Task statuses:

Status	Description
WAITING	Waiting for assignment
ASSIGNED	Assigned
RUNNING	Running
FINISHED	Finished
CANCELLED	Cancelled
FAILED	Failed
SKIPPED	Skipped

Request example:

HTTP

1 GET /task-record/page?page=1&size=10&projectId=1&status=RUNNING

Success response example:

JSON

```
1 {
2   "code": 0,
3   "message": "success",
4   "data": {
5     "records": [
6       {
7         "id": 2001,
8         "name": "Temporary task",
9         "description": "Created by third-party system",
10        "status": "RUNNING",
11        "projectId": 1,
12        "agvId": "AGV-001",
13        "createTime": "2026-04-21 10:00:00",
14        "startTime": "2026-04-21 10:01:00",
15        "endTime": null
16      }
17    ],
18    "total": 1,
19    "size": 10,
20    "current": 1
21  }
22 }
```

Failure notes: Pagination, project ID, or status parameter type errors return HTTP `500`, `code=4`; returns an empty page when no task record matches.

5.2 Query Task Action Records

- URL: `GET /action-record/list/{id}`
- Purpose: Query action details under a task record. `id` is the task record ID.

Request example:

HTTP

```
1 GET /action-record/list/2001
```

Success response example:

JSON

```
1 {  
2   "code": 0,  
3   "message": "success",  
4   "data": [  
5     {  
6       "id": 4001,  
7       "taskRecordId": 2001,  
8       "taskTemplateId": 100,  
9       "action": "PICKUP",  
10      "status": "RUNNING",  
11      "supportTypes": ["FORK-01"],  
12      "mileage": 1200,  
13      "startPoint": "P_START",  
14      "startMap": "dbe8312c60cbc75967d8b281bf4688ef",  
15      "targetPoint": "P_PICK_01",  
16      "targetMap": "dbe8312c60cbc75967d8b281bf4688ef"  
17    }  
18  ]  
19 }
```

Failure notes: Returns an empty array when the task record has no actions; `id` type errors return HTTP `500`, `code=4`.

6.Task Cancellation

6.1 Cancel a Single Task

- URL: `POST /task-record/cancel/{id}`
- Purpose: Cancel the specified task record and its unfinished actions.

Request example:

HTTP

```
1 POST /task-record/cancel/2001
```

Success response example:

JSON

```
1 {  
2   "code": 0,  
3   "message": "success"  
4 }
```

Failure notes:

- `code=2` : Task record does not exist.
- `code=401` : The current task state does not allow cancellation or the task has ended.
- HTTP `500` , `code=4` : Cancellation command delivery or status update caused an uncaught exception.

6.2 Cancel All Unfinished Tasks in a Project

- URL: `POST /task-record/all-cancel/{projectId}`
- Purpose: Cancel all unfinished tasks in the project. This API has a wide impact and should only be exposed to trusted third-party or operations systems.

Request example:

HTTP

```
1 POST /task-record/all-cancel/1
```

Success response example:

JSON

```
1 {  
2   "code": 0,  
3   "message": "success"  
4 }
```

Failure notes:

- `code=401`: Some tasks or AGV states do not allow cancellation.
- If the project has no unfinished task, the API usually returns success.

7.Error Records

7.1 Query Error Levels

- URL: `GET /error/levels`
- Purpose: Query available `level` filter values.

Request example:

HTTP

```
1 GET /error/levels
```

Success response example:

JSON

```
1 {
2   "code": 0,
3   "message": "success",
4   "data": [
5     {
6       "level": "INFO"
7     },
8     {
9       "level": "WARN"
10    },
11    {
12      "level": "ERROR"
13    }
14  ]
15 }
```

Failure notes: This API only reads built-in dictionaries and usually does not produce business failures.

7.2 Query Error Records

- URL: `GET /error/records`
- Purpose: Query project or AGV error records for third-party systems.

Request parameters:

Name	Location	Type	Required	Description
<code>page</code>	query	long	No	Default <code>1</code>
<code>size</code>	query	long	No	Default <code>10</code>
<code>projectId</code>	query	long	No	Project ID
<code>agvId</code>	query	string	No	Single AGV ID
<code>agvIds</code>	query	string[]	No	Multiple AGV IDs. Can be passed repeatedly
<code>error</code>	query	string	No	Fuzzy query by error content
<code>level</code>	query	string	No	Error level
<code>isRead</code>	query	boolean	No	Whether the error is read
<code>fromDate</code>	query	date	No	Start date
<code>targetDate</code>	query	date	No	End date

Request example:

HTTP

```
1 GET /error/records?page=1&size=10&projectId=1&agvIds=AGV-001&level=ERROR&isRead=false&fromDate=2026-04-01&targetDate=2026-04-21
```

Success response example:

JSON

```
1 {
2   "code": 0,
3   "message": "success",
4   "data": {
5     "records": [
6       {
7         "id": 1,
8         "projectId": 1,
9         "agvId": "AGV-001",
10        "level": "ERROR",
11        "error": "AGV offline",
12        "isRead": false,
13        "createTime": "2026-04-21 10:15:00"
14      }
15    ],
16    "total": 1,
17    "size": 10,
18    "current": 1
19  }
20 }
```

Failure notes: Date, boolean, or pagination parameter format errors return HTTP `500`, `code=4`; returns an empty page when no data matches.

7.3 Query Unread Error Count

- URL: `GET /error/unread/count`
- Purpose: Get the current total unread error count.

Request example:

HTTP

```
1 GET /error/unread/count
```

Success response example:

JSON

```
1 {  
2   "code": 0,  
3   "message": "success",  
4   "data": 5  
5 }
```

Failure notes: Query exceptions return HTTP `500`, `code=4`.

7.4 Mark Errors as Read

- URL: `POST /error/read/records/{projectId}`
- Purpose: After a third-party system confirms errors are handled, mark them as read. If `ids` is not passed, the API usually marks all unread errors under the project.

Request parameters:

Name	Location	Type	Required	Description
<code>ids</code>	query	long[]	No	Error record IDs. Can be passed repeatedly

Request example:

HTTP

```
1 POST /error/read/records/1?ids=1&ids=2
```

Success response example:

JSON

```
1 {  
2   "code": 0,  
3   "message": "success"  
4 }
```

Failure notes: `projectId` or `ids` type errors return HTTP `500`, `code=4`; usually returns success when no matching record exists.

7.5 Export Error Records

- URL: `GET /error/export`
- Purpose: Export error records to Excel. If the third-party system only does system-to-system data integration, prefer the JSON API in 7.2.

Request parameters:

Name	Location	Type	Required	Description
<code>projectId</code>	query	long	Yes	Project ID
<code>fromDate</code>	query	date	Yes	Start date
<code>targetDate</code>	query	date	Yes	End date
<code>sheetName</code>	query	string	No	Worksheet name
<code>agvIds</code>	query	string[]	No	AGV ID list

Request example:

HTTP

```
1 GET /error/export?projectId=1&fromDate=2026-04-01&targetDate=2026-04-21  
   &sheetName=ErrorRecords&agvIds=AGV-001
```

Success response: Excel file stream.

Failure notes:

- `code=2` : No error data matches the conditions, export failed.
- HTTP `500` , `code=4` : Date format error or export file write failure.

8.Transport and Task Statistics

8.1 Query Task Statistics Dashboard

- URL: `GET /task-statistics/{projectId}`
- Purpose: Query task count overview for the project.

Request example:

HTTP

```
1 GET /task-statistics/1
```

Success response example:

JSON

```
1 {
2   "code": 0,
3   "message": "success",
4   "data": {
5     "running": 3,
6     "finished": 20,
7     "failed": 1,
8     "cancelled": 2
9   }
10 }
```

Failure notes: `projectId` type errors return HTTP `500` , `code=4` ; returns default counts when there is no statistics data.

8.2 Query Transport Statistics

- URL: `GET /transport/statistics`

- Purpose: Aggregate transport facts by time field and dimension.

Request parameters:

Name	Location	Type	Required	Description
projectId	query	long	Yes	Project ID
fromDate	query	date	Yes	Start date
targetDate	query	date	Yes	End date
timeField	query	string	Yes	CREATED , STARTED , ENDED
groupBy	query	string	Yes	AGV , TAKE_AREA , PUT_AREA

Request example:

HTTP

1 GET /transport/statistics?projectId=1&fromDate=2026-04-01&targetDate=2026-04-21&timeField=ENDED&groupBy=AGV

Success response example:

JSON

```
1 {  
2   "code": 0,  
3   "message": "success",  
4   "data": [  
5     {  
6       "group": "AGV-001",  
7       "count": 10,  
8       "mileage": 3200  
9     }  
10  ]  
11 }
```

Failure notes: Date, enum, or project ID parameter format errors return HTTP `500`, `code=4`; returns an empty array when there is no data.

8.3 Query Transport Details

- URL: `GET /transport/data`
- Purpose: Query transport fact details.
- Request parameters: Same as 8.2.

Request example:

HTTP

```
1 GET /transport/data?projectId=1&fromDate=2026-04-01&targetDate=2026-04-21&timeField=ENDED&groupBy=AGV
```

Success response example:

JSON

```
1 {
2   "code": 0,
3   "message": "success",
4   "data": [
5     {
6       "taskRecordId": 2001,
7       "agvId": "AGV-001",
8       "takeArea": "A1",
9       "putArea": "B1",
10      "createdTime": "2026-04-21 10:00:00",
11      "endedTime": "2026-04-21 10:15:00"
12    }
13  ]
14 }
```

Failure notes: Same as 8.2.

8.4 Export Transport Statistics

- URL: `GET /transport/statistics/export`
- Purpose: Export transport statistics to Excel. If the third-party system only does system-to-system data integration, prefer the JSON API in 8.2.
- Request parameters: Same as 8.2.

Request example:

HTTP

```
1 GET /transport/statistics/export?projectId=1&fromDate=2026-04-01&target
   Date=2026-04-21&timeField=ENDED&groupBy=AGV
```

Success response: Excel file stream.

Failure notes: Same as 8.2; export write failures return HTTP `500`, `code=4`.

8.5 Export Transport Details

- URL: `GET /transport/data/export`

- Purpose: Export transport fact details to Excel. If the third-party system only does system-to-system data integration, prefer the JSON API in 8.3.
- Request parameters: Same as 8.2.

Request example:

HTTP

```
1 GET /transport/data/export?projectId=1&fromDate=2026-04-01&targetDate=2026-04-21&timeField=ENDED&groupBy=AGV
```

Success response: Excel file stream.

Failure notes: Same as 8.3; export write failures return HTTP `500`, `code=4`.